02-TECHNICAL-ARCHITECTURE.md

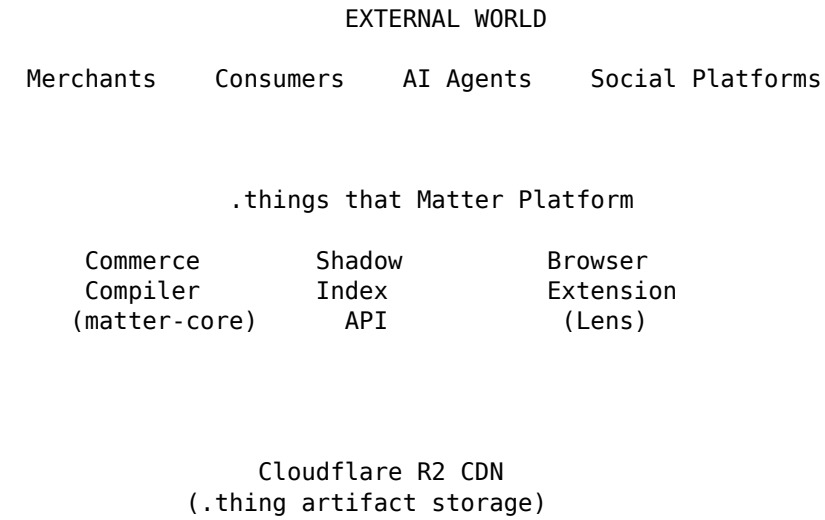
.things that Matter

Technical Architecture Document

Document Information	
Company Name	.things that Matter
Document Type	Technical Architecture
Version	1.0
Date	January 21, 2026
Author	Matome Mboweni, CTO

1. Architecture Overview

1.1 System Context



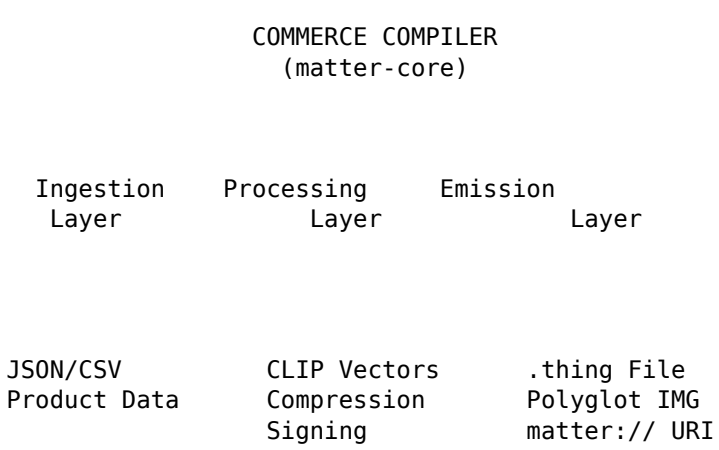
1.2 Core Design Principles

Principle	Description
Decentralization	Products are self-sovereign; no central truth server required
Efficiency	Binary format optimized for machine parsing (O(1) field access)
Portability	Artifacts travel across any medium (images, links, files)
Cryptographic Trust	Ed25519 signatures replace platform trust
Human + Agent Alloy	Dual optimization for visual appeal and computational efficiency

2. Component Architecture

2.1 Commerce Compiler (matter-core)

The heart of the system. Transforms product data into Universal Commerce Containers.



\1
- \1 Rust (primary implementation)
- \1 Google FlatBuffers
- \1 Zstandard (Zstd)
- \1 Ed25519 (via \1 crate)
- \1 Compiled for browser/Node.js via \1

\1

Operation	Description	Output
`compile()`	Full pipeline: ingest → process → emit	.thing + image + URI
`sign()`	Apply Ed25519 signature	Signed .thing
`verify()`	Validate signature integrity	Boolean
`vectorize()`	Generate CLIP embedding	512-dim float vector
`embed()`	Steganographic image embedding	Polyglot image

2.2 FlatBuffers Schema

Binary format for Universal Commerce Containers.

```
// thing.fbs - Universal Commerce Container Schema

namespace Matter;

table Product {
  // Identity
  id: string (required);
  hash: [ubyte] (required);        // SHA-256 of canonical form
  version: uint8 = 1;

  // Commerce Data
  title: string (required);
  description: string;
  price: uint64;                   // Price in cents (avoid floats)
  currency: string = "USD";
  sku: string;
```

```
  // Media
  image_url: string;
  thumbnail: [ubyte];              // Embedded thumbnail (optional)

  // Semantic Layer (Cognitive)
  clip_vector: [float];            // 512-dim CLIP embedding
  categories: [string];
  tags: [string];

  // Commerce Logic
  checkout_url: string (required);
  merchant_id: string;
  inventory_hint: InventoryStatus = UNKNOWN;

  // Metadata
  created_at: uint64;              // Unix timestamp
  expires_at: uint64;              // TTL for freshness
  source_url: string;

  // Cryptographic Seal
  signature: [ubyte];              // Ed25519 signature
  signer_pubkey: [ubyte];          // Public key for verification
}

enum InventoryStatus : byte {
  UNKNOWN = 0,
  IN_STOCK = 1,
  LOW_STOCK = 2,
  OUT_OF_STOCK = 3
}

table ThingFile {
  magic: uint32 = 0x54484E47;      // "THNG" magic bytes
  version: uint8 = 1;
  product: Product (required);
  gist: Gist;                      // Minimal data for quick reads
}

table Gist {
  title: string;
  price: uint64;
  currency: string;
  image_url: string;
}

root_type ThingFile;
```

\1

Feature	JSON	Protobuf	FlatBuffers
Parse time	O(n)	O(n)	**O(1)**
Memory allocation	High	Medium	**Zero-copy**
Random field access	No	No	**Yes**
Human readable	Yes	No	No
Schema evolution	Poor	Good	Good

\1 AI agents can read \1 directly from byte offset without parsing entire file.

2.3 Semantic Vectorization (CLIP Integration)



```
   Product Image
                    CLIP ViT-B/32  512-dim Vector
   Product Title


                    Embedded in .thing
                    (Cognitive Layer)

   Use Cases:
   • "Find shoes with this vibe" (semantic search)
   • Similar product recommendations
   • Zero-shot categorization
```

\1
- \1 Replicate API (serverless CLIP)
- \1 ViT-B/32 (OpenAI CLIP)
- \1 ~R0.40 per image
- \1 ~200ms per inference

\1
- Embedded directly in .thing file header
- Enables local similarity search without API calls
- 512 floats × 4 bytes = 2KB per product

2.4 Cryptographic Signing

```
                   ED25519 SIGNING


   Compiler Private Key (kept secret)



     Sign(payload)    64-byte signature


   Verification (client-side):



    Verify(payload,
      signature,      Boolean (valid/invalid)
      pubkey)

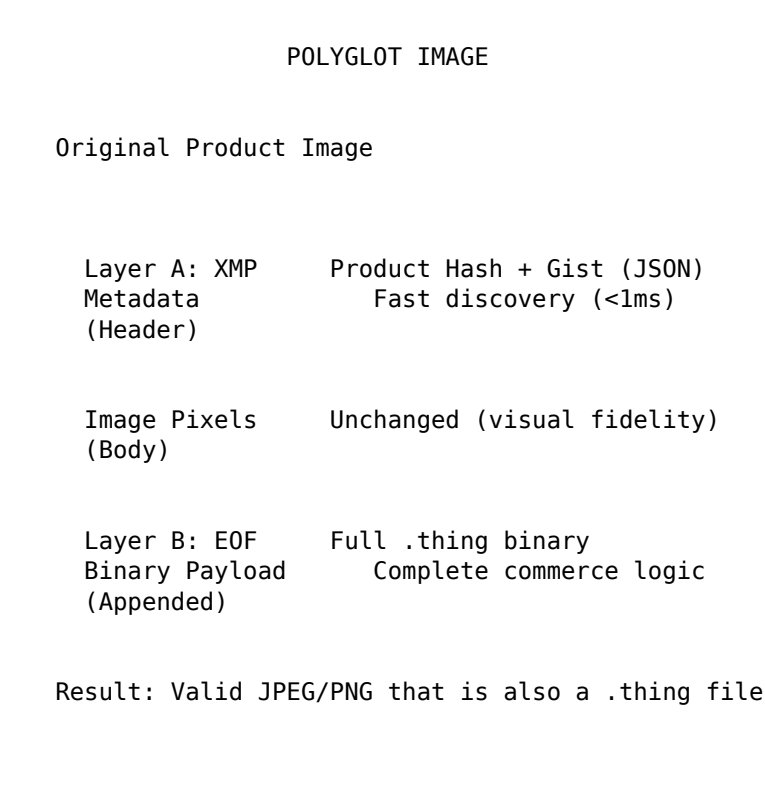

   If ANY byte changes  Signature fails
   Price tampering impossible without private key
```

\1
- Fast verification (important for browser extension)
- Small signature size (64 bytes)
- Deterministic signatures
- Well-audited, no ECDSA pitfalls

\1
- Private key stored in secure environment variable
- Public key distributed with extension/SDK
- Key rotation: new key = new signature version

2.5 Distribution Matrix

2.5.1 Polyglot Image (Steganography)



\1

```
<x:xmpmeta xmlns:x="adobe:ns:meta/">
  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
    <rdf:Description rdf:about=""
      xmlns:matter="https://thingsthatmatter.com/ns/1.0/">
      <matter:hash>a1b2c3d4...</matter:hash>
      <matter:gist>{"title":"Product","price":2999,"currency":"USD"}</matter:gist>
      <matter:version>1</matter:version>
    </rdf:Description>
  </rdf:RDF>
</x:xmpmeta>
```

\1

Platform	XMP Survives	EOF Survives	Strategy
Discord	Yes	Yes	Full polyglot works
Telegram	Yes	Yes	Full polyglot works
Twitter/X	No	No	Use matter:// URI
Instagram	No	No	Use matter:// URI
WhatsApp	Partial	No	XMP gist + matter://
Email	Yes	Yes	Full polyglot works
Direct download	Yes	Yes	Full polyglot works

2.5.2 Matter URI Protocol

```
matter://a1b2c3d4e5f6...

Resolution Flow:
1. Extension intercepts matter:// link
2. Extract hash from URI
3. Query Shadow Index API: GET /resolve/{hash}
4. Receive CDN URL for .thing file
5. Fetch and verify .thing
6. Display product overlay
```

\1

```
matter://{base58_hash}[?params]

Examples:
matter://QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG
matter://QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG?ref=twitter
```

2.5.3 Raw .thing File

Direct file distribution for:
- Developer integrations
- Bulk data exchange
- Offline catalogs
- AI agent consumption

\1 \1
\1 \1

2.6 Shadow Index API

Fast hash-to-artifact resolution service.

```
                    SHADOW INDEX API


   Endpoints:

   POST /compile
   Body: { product_json }
   Response: { hash, thing_url, image_url, matter_uri }

   GET /resolve/{hash}
   Response: { thing_url, gist, expires_at }

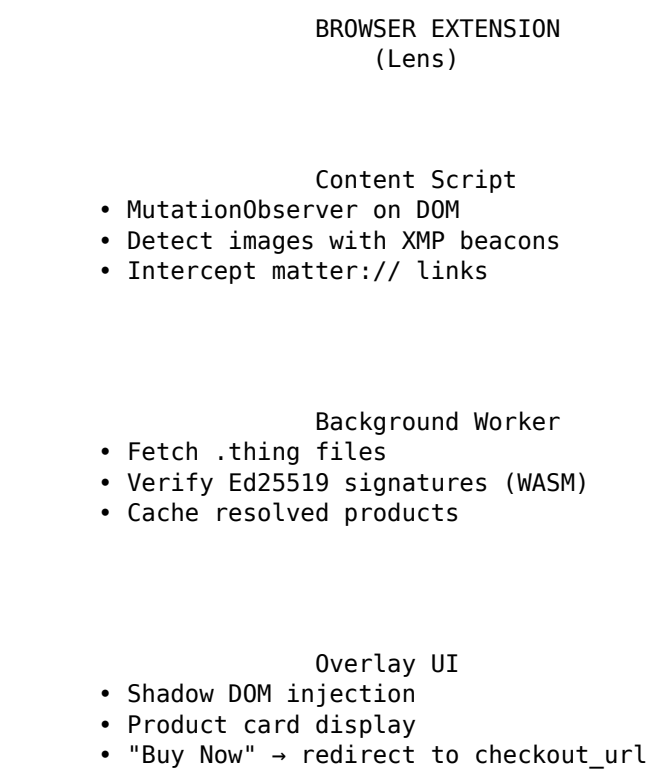
   GET /gist/{hash}
   Response: { title, price, currency, image_url }

   POST /batch
   Body: { products: [...] }
   Response: { results: [...] }
```

\1

- \1 Vercel Edge Functions (free tier)
- \1 Supabase (hash → metadata mapping)
- \1 Cloudflare R2 (artifact files)
- \1 Redis Cloud free tier (hot lookups)

2.7 Browser Extension (Lens)



\1
- \1 Vanilla JS (minimize bundle size)
- \1 libmatter compiled to WASM
- \1 Shadow DOM (isolated styles)
- \1 IndexedDB (local cache)

3. Data Flow

3.1 Compilation Flow (Merchant → Artifact)

```
1. Merchant submits product JSON


2. Compiler validates input schema


3. Fetch product image, send to CLIP API


4. Receive 512-dim semantic vector
```

```
5. Compress description with Zstd

6. Serialize to FlatBuffers binary

7. Sign with Ed25519 private key

8. Generate polyglot image (XMP + EOF)

9. Upload artifacts to Cloudflare R2

10. Index hash in Shadow Index DB

11. Return: { hash, thing_url, image_url, matter_uri }
```

3.2 Discovery Flow (Consumer finds product)

```
1. Consumer sees image on social media

2. Lens extension detects XMP beacon

3. Extract hash from beacon

4. Check local cache for .thing
          Cache hit: Use cached data
          Cache miss: Fetch from CDN

5. Verify Ed25519 signature
          Invalid: Show warning, block
          Valid: Continue

6. Render product overlay (Shadow DOM)

7. User clicks "Buy Now"

8. Redirect to merchant checkout_url
```

4. Security Architecture

4.1 Threat Model

Threat	Attack Vector	Mitigation
Price tampering	Modify .thing file	Ed25519 signature invalidates
Fake products	Create malicious .thing	Only signed by our key accepted
Man-in-middle	Intercept CDN response	HTTPS + signature verification
XSS via overlay	Inject malicious HTML	Shadow DOM isolation, CSP
Key compromise	Steal signing key	Key rotation, monitoring

4.2 Trust Model

```
Trust Anchor: .things that Matter signing key


Verified: Compiler signs all .thing files


Client: Extension only trusts our public key

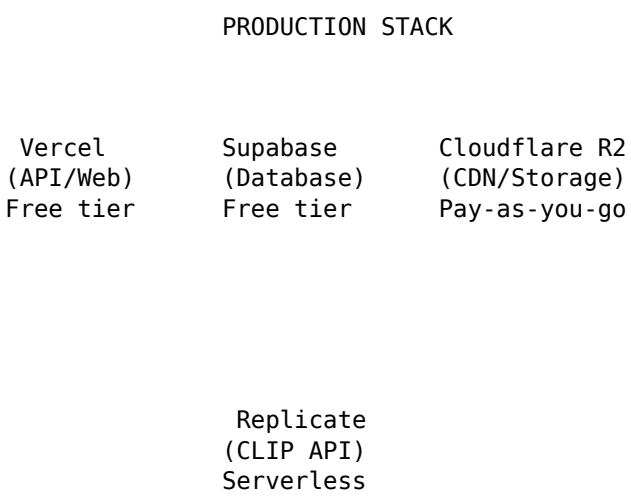

Result: Tamper-proof commerce data
```

4.3 Security Controls

Control	Implementation
Input validation	Zod schema validation
Output encoding	Strict FlatBuffers schema
Cryptographic signing	Ed25519
Transport security	HTTPS everywhere
DOM isolation	Shadow DOM for overlays
Content Security Policy	Strict CSP in extension

5. Infrastructure

5.1 Production Architecture



5.2 Service Dependencies

Service	Purpose	Tier	Monthly Cost
Vercel	API hosting, web app	Hobby (Free)	R0
Supabase	Database, auth	Free	R0
Cloudflare R2	Artifact storage	Pay-as-you-go	~R630/mo
Replicate	CLIP inference	Pay-per-use	~R540/mo
Redis Cloud	Caching	Free	R0
Sentry	Error tracking	Free	R0

6. Development Standards

6.1 Code Organization

```
things-that-matter/
 packages/
    matter-core/          # Rust compiler library
       src/
          lib.rs
          schema/       # FlatBuffers definitions
          compiler/     # Compilation pipeline
          crypto/       # Ed25519 signing
          steg/         # Steganography engine
          clip/         # CLIP integration
       Cargo.toml
       README.md

    matter-ffi/           # FFI bindings (C/C++/Node)
       bindings/
       Cargo.toml
       README.md

    shadow-index/         # API service
       api/
       package.json
       README.md

    lens-extension/       # Browser extension
        src/
        manifest.json
        README.md

 docs/                     # Documentation
 scripts/                  # Build/deploy scripts
 README.md
```

6.2 Technology Choices

Component	Technology	Rationale
Compiler	Rust	Memory safety, WASM compilation, performance
API	TypeScript/Node	Vercel compatibility
Extension	Vanilla JS	Minimal bundle size
Schema	FlatBuffers	Zero-copy access
Storage	Cloudflare R2	S3-compatible, cheap egress
Database	PostgreSQL (Supabase)	Reliable, free tier

6.3 Quality Standards

Metric	Target
Test coverage	>70% for compiler
Documentation	All public APIs documented
Code review	Self-review + open-source community
Performance	<500ms compile time
Bundle size	<100KB extension

7. Appendices

7.1 API Reference (Shadow Index)

See separate API documentation.

7.2 FlatBuffers Schema

See \1 in repository.

7.3 Extension Manifest

See \1 in repository.

\1 1.0
\1 January 21, 2026
\1 Post-Phase 0 (March 2026)